

Hilos

Módulo 4

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"

Hilos

- Revisión
- Modelos Multihilados
- Librerías de Hilos
- Aspectos sobre Hilos
- Ejemplos de Sistemas Operativos
- Hilos en Windows XP
- Hilos en Linux

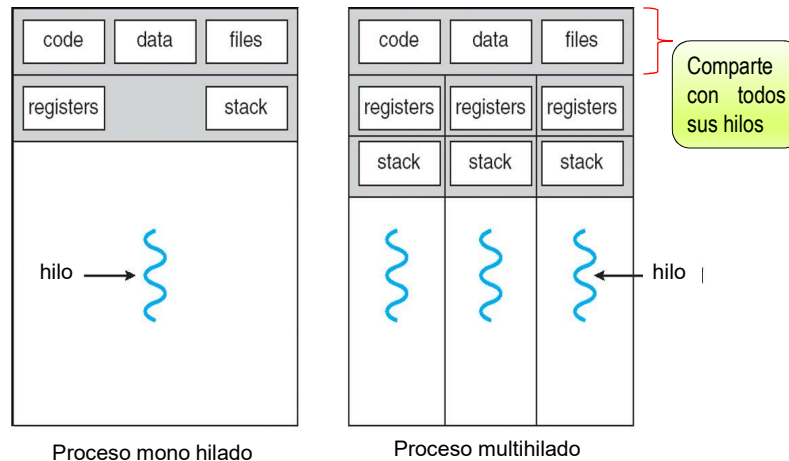
Hilos

- Un *thread* (o *proceso de peso liviano*) es una unidad básica de utilización de CPU, consiste de:
 - ID
 - contador de programa
 - conjunto de registros
 - espacio de stack
- Comparte con otros thread compañeros que pertenecen al mismo proceso, su:
 - sección de código
 - sección de datos
 - recursos del SO.
- Un proceso tradicional o *peso pesado*, es igual a una tarea (task) con un solo hilo de control.

Hilos

- Con un proceso multi-threads, mientras un thread está bloqueado y esperando, un segundo thread del mismo proceso puede estar **corriendo**
 - Un procesador de texto puede tener un hilo para mostrar gráficos, otro para responder pulsaciones del teclado del usuario y un tercer hilo para corrector ortográfico.
- La cooperación de múltiples threads sobre una misma tarea ofrece alto procesamiento total y mejora el rendimiento.
 - Aplicaciones que requieren compartir un buffer común (p.e., productor-consumidor) se benefician con la utilización de threads.
- Los threads permiten paralelismo al combinar ejecución secuencial y llamadas a sistemas bloqueantes (por ejemplo que requiera E/S).

Procesos Mono y Multihilos



JRA © - LRS 2025

Sistemas Operativos – Hilos

Beneficios

- Capacidad de Respuesta
 - En una aplicación interactiva, mientras atiende al usuario, permite que otro thread continúe ejecutando.
- Compartir Recursos
 - Dado que los *threads* dentro de un mismo proceso comparten memoria y archivos, pueden comunicarse unos con otros sin invocar al kernel
- Economía
 - Toma menos tiempo crear un nuevo *thread* que un proceso
 - Menos tiempo terminar un *thread* que un proceso
 - Menos tiempo en conmutar entre dos *threads* dentro del mismo proceso (5 veces)
- Utilización sobre Arquitecturas Multiprocesador: aumenta paralelismo porque los hilos pueden ejecutarse en distintas CPUs.

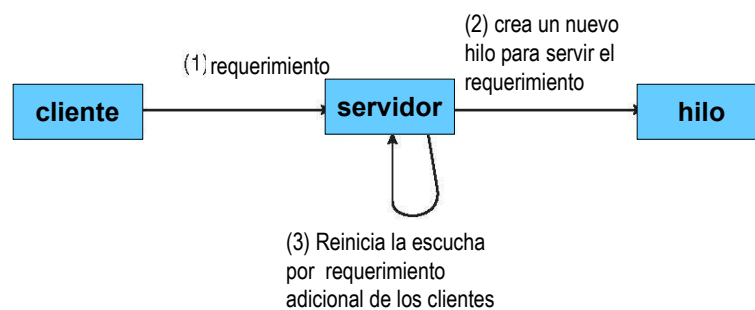
JRA © - LRS 2025

Sistemas Operativos – Hilos

Programación Multicore

- Los sistemas multicore ponen presión sobre los programadores, estos desafíos incluyen:
 - Dividir el programa en actividades que sean paralelas
 - Balance de carga (como distribuir la ejecución entre distintos core)
 - Partición de datos
 - Dependencia de los datos
 - Verificación y depuración

Arquitectura de Servidor Multihilado

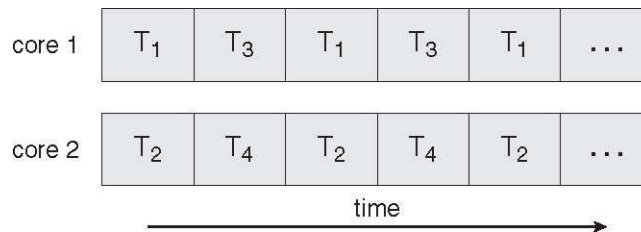


Ejecución Concurrente y paralela

Un solo cpu. Concurrente.



Múltiples núcleos. Paralela



JRA © - LRS 2025

Sistemas Operativos – Hilos

Funcionalidad de los Hilos

- **Estado** de los hilos: **ejecutando, listo y bloqueado**
- Operaciones básicas asociadas con un **cambio de estado** del hilo
 - Creación
 - Bloqueo ¿implica el bloqueo del proceso completo?
 - Desbloqueo
 - Finalización
- Sincronización de hilos →
 - Todos los hilos de un proceso comparten el mismo espacio de direcciones y otros recursos (x ej archivos abiertos)
 - Es necesario utilizar técnicas para que no interfieran entre ellos. Las mismas que en los procesos.

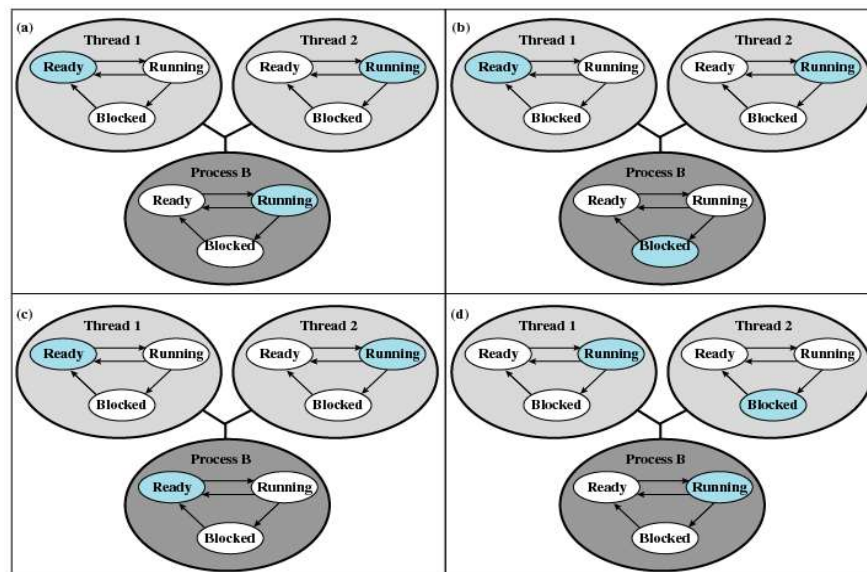
JRA © - LRS 2025

Sistemas Operativos – Hilos

Hilos a Nivel de Usuario

- La gestión de hilos se realiza a través de **librerías** a nivel de usuario que me permite:
 - Crear/destruir
 - Paso de mensajes y datos entre threads
 - Planificar la ejecución
 - Salvar y restaurar el **contexto** de los hilos
- Tres principales librerías de hilos actualmente en uso:
 - POSIX **Pthreads**
 - Win32 threads
 - Java threads

Hilos a Nivel de Usuario



Hilos a Nivel de Usuario

- 1) La aplicación corriendo en el thread 2 hace una llamada a sistema que bloquea B (por ejemplo: E/S). Esto transfiere el control al kernel que pone al proceso B en estado bloqueado (espera) pero de acuerdo a la estructura de datos mantenida por la librería de threads, el thread 2 permanece en “corriendo” (b)
- 2) Cuando el proceso B ha agotado su tiempo pasa al estado de listo, pero de acuerdo a la estructura de datos mantenida por la librería de threads, el thread se mantiene en estado “corriendo”. (c)
- 3) El thread 2 ha alcanzado un punto donde necesita alguna acción por parte del thread 1. El thread 2 entra en estado de bloqueado y el thread 1 pasa a “corriendo”. El proceso B que los contiene se mantiene en estado “corriendo” (d)

JRA © - LRS 2025

Sistemas Operativos – Hilos

Hilos a Nivel de Usuario

- Ventajas hilos a nivel Usuario(ULT) vs hilos a nivel kernel (KLT)
 - Un cambio de hilo no requiere cambio de modo
 - Planificador a medida de la aplicación
 - Los ULT no necesitan un kernel especial que les de soporte para su ejecución.
- Desventajas
 - Llamadas a sistemas bloqueantes
 - No puede sacar ventaja del multiprocesador; el SO asigna el proceso a un solo procesador al mismo tiempo.

JRA © - LRS 2025

Sistemas Operativos – Hilos

Hilos a Nivel de Kernel

- Son soportados por el Kernel
- Se usa una API p/ acceder a las utilidades de hilos del kernel.
- Todos los hilos de una aplicación se mantienen en un solo proceso.
- El kernel mantiene información del contexto del proceso y de los hilos individuales del proceso.
- Planificación la realiza el SO y es a nivel de hilo.
- Ejemplos
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X

JRA © - LRS 2025

Sistemas Operativos – Hilos

Hilos a Nivel de Kernel

- Ventaja
 - El kernel puede planificar multiples hilos en multiples procesadores
 - Si se bloquea un hilo puede planificar otro del mismo proceso
- Desventaja
 - La transferencia de control entre hilos del mismo proceso requiere cambio de modo.

JRA © - LRS 2025

Sistemas Operativos – Hilos

Modelos de Multihilados

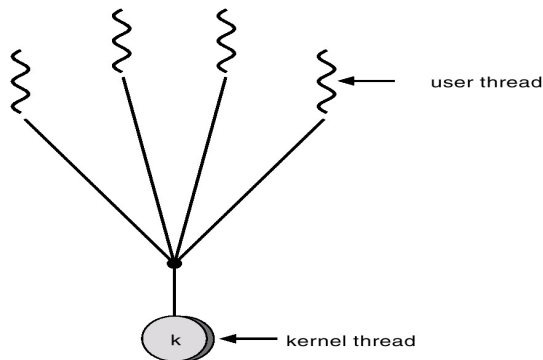
- Muchos-a-Uno
- Uno-a-Uno
- Muchos-a-Muchos

JRA © - LRS 2025

Sistemas Operativos – Hilos

Muchos-a-Uno

- ➡ Muchos hilos a nivel de usuario mapean a un hilo a nivel kernel.
- ➡ Usado en sistemas que no soportan hilos a nivel kernel.

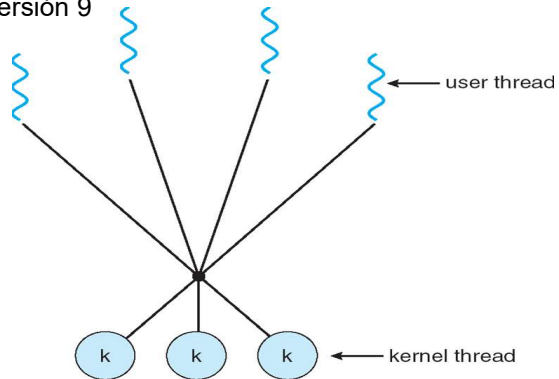


JRA © - LRS 2025

Sistemas Operativos – Hilos

Muchos-a-Muchos

- Permite que muchos hilos a nivel de usuario mapeen a muchos hilos a nivel de kernel
- Permite al SO crear un número suficiente de hilos a nivel de kernel
- Windows NT/2000 con paquete *ThreadFiber*
- Solaris antes de la versión 9



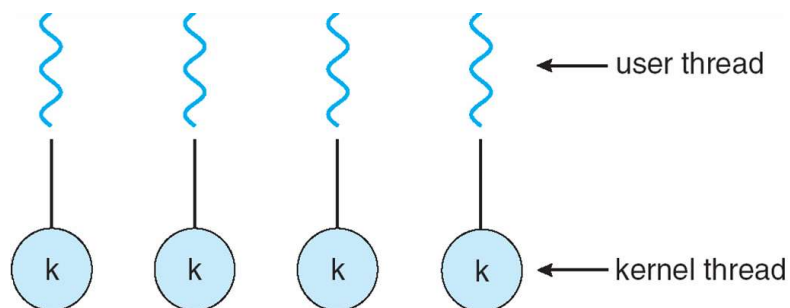
JRA © - LRS 2025

Sistemas Operativos – Hilos

Uno-a-Uno

- Cada thread nivel usuario mapea a un thread kernel.
- Ejemplos

- Familia de SO Windows
- Linux
- Solaris 9 en adelante



JRA © - LRS 2025

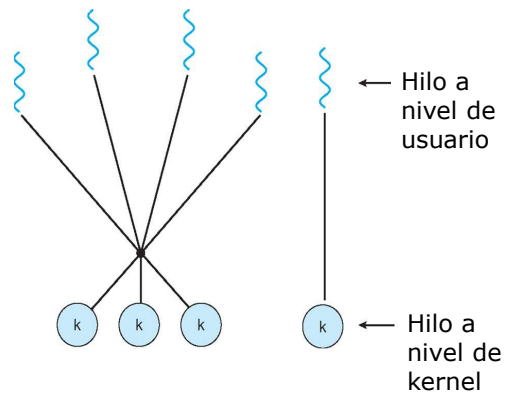
Sistemas Operativos – Hilos

Modelo de Dos Niveles

- Similar a M:M, excepto que permite a un hilo a nivel de usuario ser **limitado** a un hilo a nivel de kernel

- Ejemplos

- HP-UX
- Tru64 UNIX
- Solaris 8 y anteriores



JRA © - LRS 2025

Sistemas Operativos – Hilos

Librerías de Hilos

- Las librerías de hilos proveen a los programadores APIs para crear y administrar hilos
- Dos formas de implementarlas
 - Librerías enteramente en espacio de usuario
 - Librería a nivel de Kernel soportada por el SO

JRA © - LRS 2025

Sistemas Operativos – Hilos

Pthreads

- Pueden ser provistas tanto a nivel de usuario como a nivel de kernel
- Es un standard POSIX (IEEE 1003.1c) define la API para creación y sincronización de hilos
- Las API especifican el comportamiento de la librería de hilos
- Común en SOs UNIX (Solaris, Linux, Mac OS X)

Aspectos sobre hilos

Semantica de fork() y exec()

- La llamada fork() crea un proceso duplicado.
- Con hilos ¿Duplica el **fork()** solo el hilo invocador o todos los hilos del nuevo proceso?
- Unix dispone de 2 versiones de fork():
 - Si un hilo del programa llama a fork() y luego a exec(), duplica solo el hilo que hace la llamada
 - Si **no llama exec()** luego del fork(), el nuevo proceso deberá duplicar todos los hilos.

Cancelación de Hilos

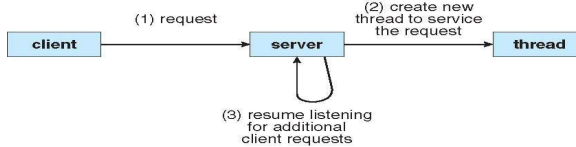
- Implica terminar un hilo antes que se haya completado
 - Ej: La búsqueda concurrente en una Base de Datos y un hilo devuelve el resultado
- Dos propuestas generales:
 - **Cancelación asincrónica** termina inmediatamente el hilo señalado
 - **Cancelación Diferida** permite al hilo señalado verificar periódicamente si debería ser cancelado. Permite que la cancelación se pueda hacer de forma segura.

Manejo de Signal

- Los *Signals* son usados en UNIX para **notificar** a un proceso que ha ocurrido un evento particular
- Los signals siguen un mismo patron:
 1. es **generado por un suceso** particular
 2. es **enviado** a un proceso
 3. una vez enviado, debe **ser tratado**
- *Tratamiento de señales* → *signals handler*
- *Si tengo un proceso con varios hilos ¿A quien enviar la señal?*
Opciones:
 - Enviar la señal al hilo sobre el cual se aplica el signal
 - Enviar la señal a cada hilo del proceso
 - Enviar la señal a ciertos hilos del proceso
 - Asignar un hilo específico para recibir todas las señales del proceso

Pools de Hilos

- Ejemplo del servidor web con hilos.



Cuando el servidor recibe un requerimiento, crea un hilo nuevo para dar servicio, esto insume **tiempo creacion/ destrucción del hilo**.

Además el numero de hilos activos son **recursos limitados**.

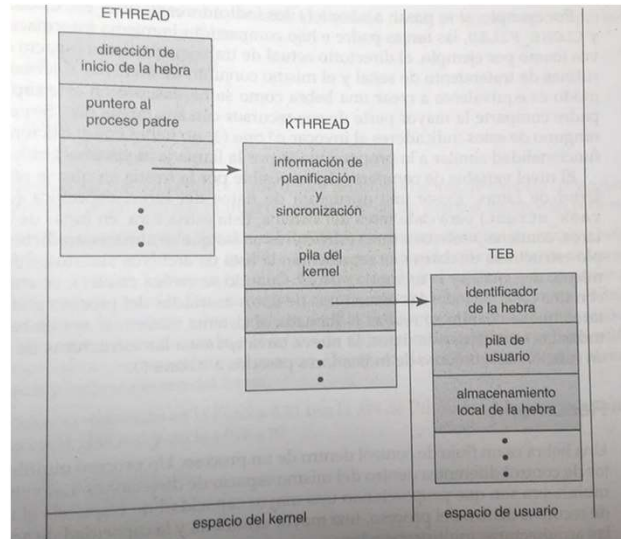
Una es Solución:

- Crear un número de hilos en un pool (*conjunto compartido*) donde esperan por trabajo.
- Ventajas:
 - Es mas rápido servir un requerimiento con un hilo existente que crear uno nuevo
 - Permite que el número de hilos de la aplicación sea limitado al tamaño del pool

Hilos Windows

- Implementa un mapeo uno-a-uno con los hilos a nivel de kernel
- Cada hilo contiene
 - Una identificación
 - Un conjunto de registros (estado del procesador)
 - Pilas separadas de usuario y kernel
 - Área de datos privados, usada por las distintas librerías de tiempo de ejecución y de enlace dinámico DLL
- El conjunto de registros, pilas y área privada de datos constituyen el **contexto** de los hilos
- La estructura de datos de un hilo incluye:
 - ETHREAD (bloque ejecutivo del hilo)
 - KTHREAD (bloque de hilo en el kernel)
 - TEB (bloque de entorno del hilo)

Estructura de un hilo en Windows XP



JRA © - LRS 2025

Sistemas Operativos – Hilos

Hilos en Linux

- Linux se refiere a ellos como *tareas* mas que como *hilos*
- La creación de hilos es hecha por la llamada a sistema **clone()**
- **clone()** permite a una tarea hija compartir el espacio de direcciones de la tarea-padre (proceso)
- Cuando se invoca a clone(), se pasa un conjunto de *indicadores* para determinar el nivel de compartición

indicador	significado
CLONE_FS	Se comparte información del sistema de archivos.
CLONE_VM	Se comparte el mismo espacio de memoria.
CLONE_SIGHAND	Se comparten los descriptores de señal.
CLONE_FILES	Se comparte el conjunto de archivos abiertos.

JRA © - LRS 2025

Sistemas Operativos – Hilos

Fin

Módulo 4

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"